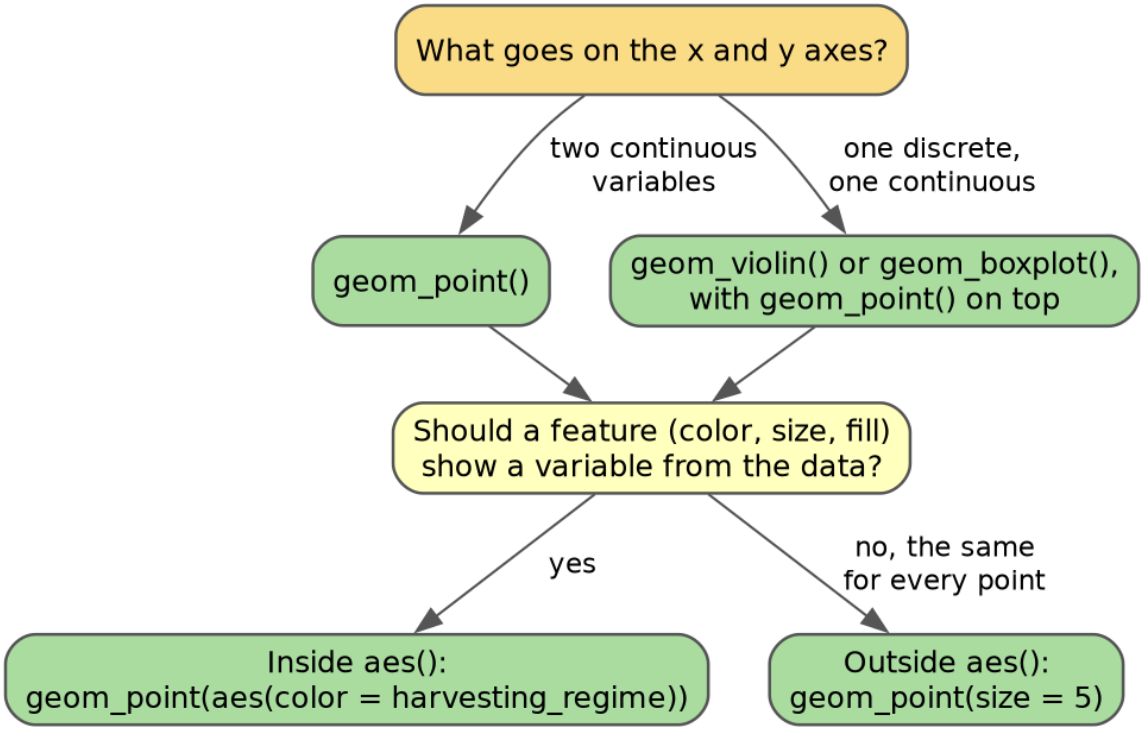
Chapter summaries

Integrated Bioanalytical Chemistry · thebustalab.github.io/integrated_bioanalytics

1. data visualization I · page 2
2. data visualization II · page 3
3. wrangling and summaries · page 4
4. distances and scaling · page 5
5. networks · page 6
6. hierarchical clustering · page 7
7. flat clustering · page 8
8. dimensional reduction · page 9
9. comparing groups · page 10
10. regression models · page 11
11. analysis of variance · page 12
12. machine learning models · page 13
13. language models · page 14
14. protein language models · page 15
15. homology · page 16
16. alignments · page 17
17. phylogenies · page 18
18. phylogenetic analyses · page 19

data visualization I

Integrated Bioanalytical Chemistry · thebustalab.github.io/integrated_bioanalytics

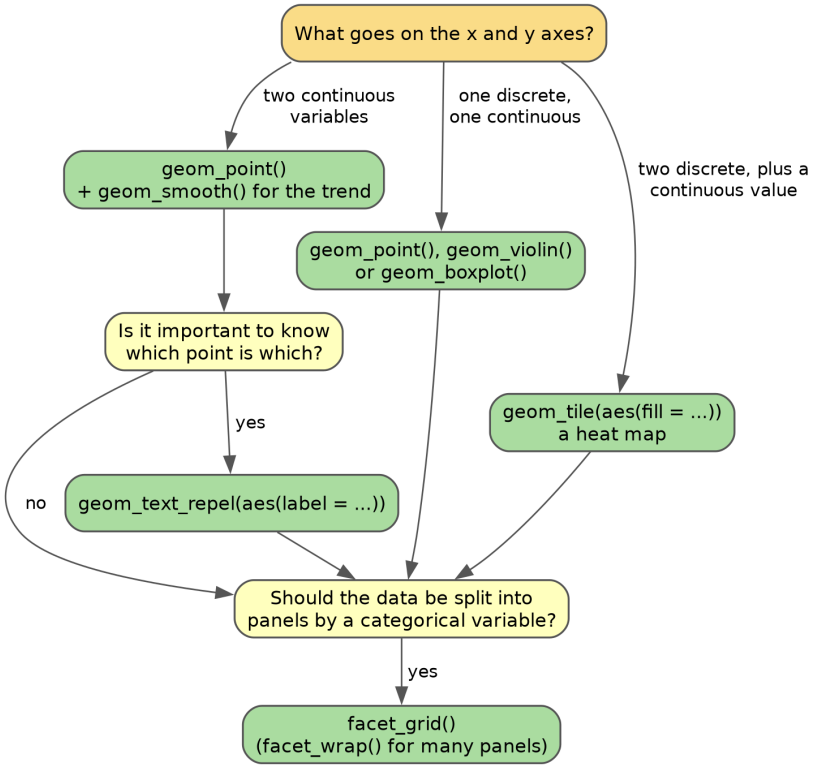


Objects store data (like: `my_object <- 7`), and functions act on it: `filter()` keeps only the rows that meet a logical condition, such as `filter(my_data, chemical_species == "FAs")`. A ggplot is then built in three steps: name the data with `ggplot(data = ...)`, map variables onto the axes with `aes()`, and add one or more geoms with +. Geoms are geometric objects with features (size, color, shape, etc.). A feature mapped to a **variable** in the data goes **inside** `aes()` (`geom_point(aes(color = harvesting_regime))`), and a feature set to a **constant** goes **outside** `aes()` (`geom_point(size = 5)`). Swapping mappings around is cheap, so it is worth trying a few to see which plot best communicates the aspect of your data that you are interested in.

Function or operator	What it does	What we vary
`<-`	stores a value or a result in an object	the object name on the left
`filter()`	keeps only the rows that meet a condition	the condition: ==, !=, <, <=, >, >=, `%in% c(...)`
`ggplot()`	starts a plot and names the data	`data =`
`aes()`	maps variables onto the axes and onto features of the shapes	x, y, `color`, `size`, `fill`
+	adds a geom to the plot, and several geoms can be stacked	end the line with + to continue on the next
`geom_point()`	draws one point per observation	`aes(color = ...)`, `aes(size = ...)`, or constants such as `size = 5`, `color = "black"`
`geom_violin()`	draws the spread of values in each group	`aes(fill = ...)`
`geom_boxplot()`	draws a box plot for each group	nothing extra in this chapter

data visualization II

Integrated Bioanalytical Chemistry · thebustalab.github.io/integrated_bioanalytics

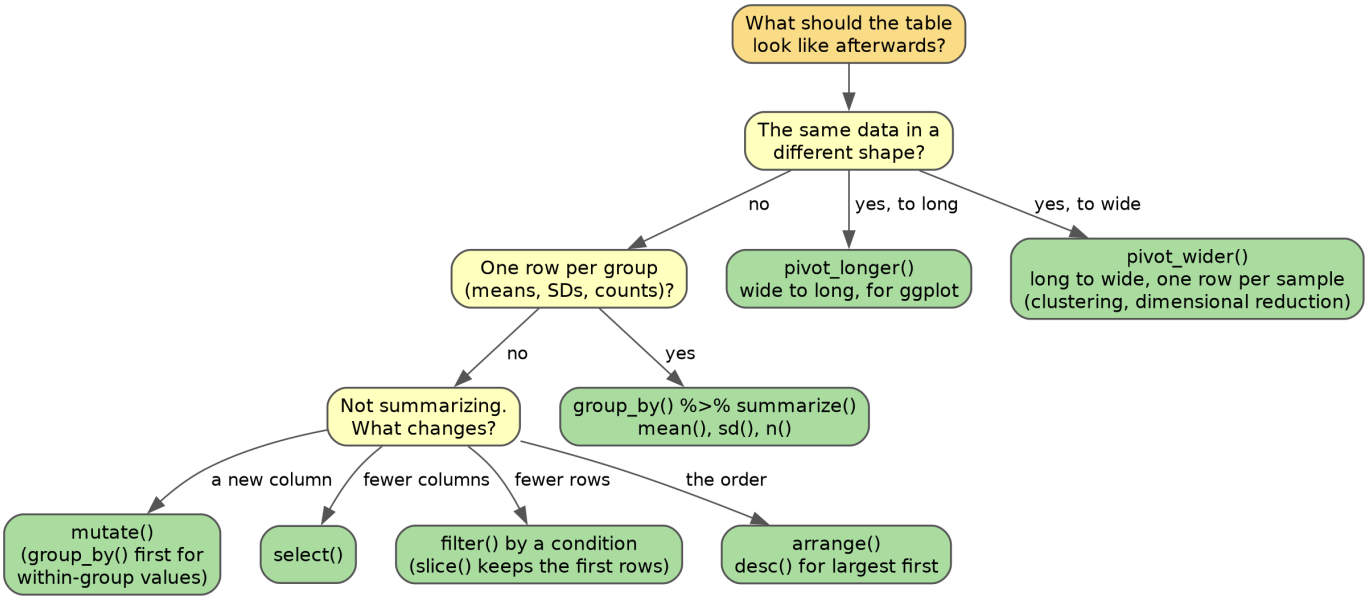


The kind of data being plotted can help you choose which geom to use: `geom_smooth()` with `geom_point()` for two continuous variables, and `geom_tile()` (a heat map) for two discrete variables and one continuous variable. When labeling which point is which matters, `geom_text_repel()` labels them without overlap. Facets split a plot into small multiples, one panel per level of a categorical variable. Every aesthetic mapping creates a scale that a `scale_*` command adjusts, themes control the non-data ink, and `plot_grid()` combines finished plots into one figure.

Function	What it is for	What we vary
`geom_smooth()`	a trend line through two continuous variables	`color`
`geom_tile()`	a heat map: two discrete variables and one continuous	`aes(fill = ...)`, border `color`, `linewidth`
`geom_text()`, `geom_text_repel()`	text at each point (the repel version nudges labels apart)	`aes(label = ...)`
`facet_grid()`	small multiples in a row, a column, or both	`. ~ var`, `var ~ .`, `var1 ~ var2`
`facet_wrap()`	small multiples when there are many panels	`var ~ .`
`scale_fill_gradient()`	the color gradient of a continuous fill	`low`, `high`
`scale_fill_brewer()`	RColorBrewer colors for a discrete fill	`palette`
`scale_x_continuous()`, `scale_y_continuous()`	axis title, tick marks and range	`name`, `breaks`, `limits`
`theme_classic()`, `theme_bw()`, `theme_dark()`, `theme_void()`	complete themes for the non-data ink	which one
`theme()`	individual non-data components	`text = element_text(size = ..., color = ...)`, `axis.text`
`plot_grid()`	arranges finished plot objects into one figure	the plot objects, `ncol`

wrangling and summaries

Integrated Bioanalytical Chemistry · thebustalab.github.io/integrated_bioanalytics

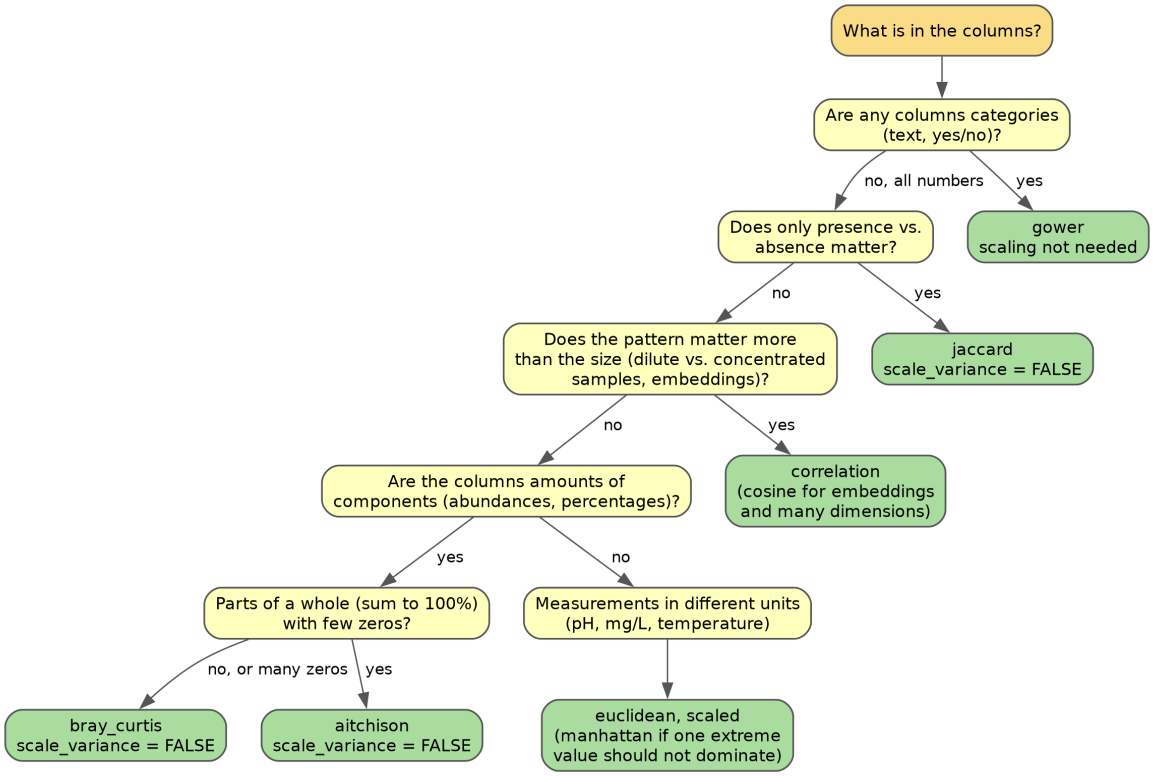


Wrangling comes down to deciding what the table should look like after each step. Pivoting changes the shape without changing the data: long tables suit ggplot, and wide tables with one row per sample suit the matrix analyses later in the book. `group_by()` with `summarize()` collapses each group to a single row, while `mutate()` keeps every row and adds a column. The pipe (`%>%`) chains these steps together, and `view()` on the result is the quickest way to check that each step did what was intended.

Function	What it does	What to set
`view()`	opens a whole table as a scrollable grid, with each column's type	the table, plus `n = 5000` to see past the first 1000 rows (`View()` in RStudio)
`select()`	keeps only the named columns	the column names
`filter()`	keeps only the rows that meet a condition	the condition, such as `park == "BELA"`
`pivot_longer()`	wide to long: column names and values each become a new column	`cols`, `names_to`, `values_to`
`pivot_wider()`	long to wide, the exact inverse of `pivot_longer()`	`names_from`, `values_from`
`%>%`	passes the result on the left to the function on the right as its first argument	
`group_by()` + `summarize()`	one row per group, holding the statistics asked for	the grouping columns, with `mean()`, `sd()`, `n()` inside `summarize()`
`arrange()`	sorts rows, smallest first	the column, wrapped in `desc()` for largest first
`slice()`	keeps the first n rows (of each group, after `group_by()`)	n, as in `slice(1)`
`mutate()`	adds a new column and keeps every row, and works within groups after `group_by()`	the new column, as in `percent_S = mg_per_L/sum(mg_per_L)*100`
`geom_pointrange()`	plots a summary point with a range around it	`x` and `y`, plus `ymin` and `ymax` (or `xmin` and `xmax`)

distances and scaling

Integrated Bioanalytical Chemistry · thebustalab.github.io/integrated_bioanalytics

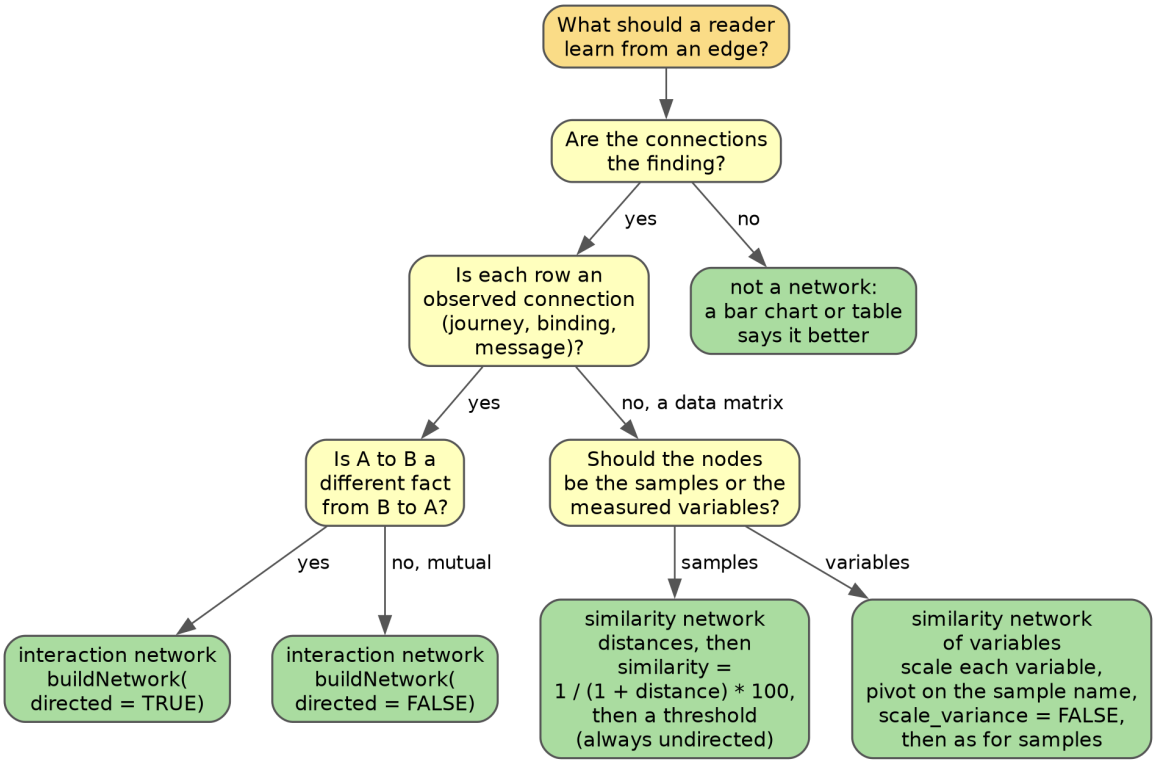


Two decisions go into every distance matrix. **Scaling decides which variables count**: scale measurements that are in different units so that no single variable dominates, and leave raw amounts unscaled when you want distances to reflect potential differences in the scales that your variables occupy. **The distance metric decides what "similar" means**: the same size and values, the same pattern, the same composition, the same things present, or the same mix of measurements and categories. The flowchart and the table below summarize the choices.

`distance_method`	Similar means...	Typical data	Scale first?
`"euclidean"`	close values in every variable, with large single differences counting heavily	measurements in different units (pH, mg/L, temperature)	yes, when units differ
`"manhattan"`	close values in every variable, with one extreme difference counting less	as euclidean, when outliers or spikes are a concern	yes, when units differ
`"correlation"`	the same pattern across the variables, regardless of size	profiles at different concentrations or totals	optional
`"cosine"`	pointing in the same direction, regardless of length	embeddings and other high-dimensional data	optional
`"bray_curtis"`	sharing the same amounts of the same components	abundances and percentages, especially with many zeros	no
`"aitchison"`	the same proportions between components	compositions summing to 100% with few zeros	no
`"jaccard"`	containing the same things, amounts ignored	presence/absence, detected or not	no
`"gower"`	matching across a mix of numbers and categories	tables with categorical columns	handled internally

networks

Integrated Bioanalytical Chemistry · thebustalab.github.io/integrated_bioanalytics

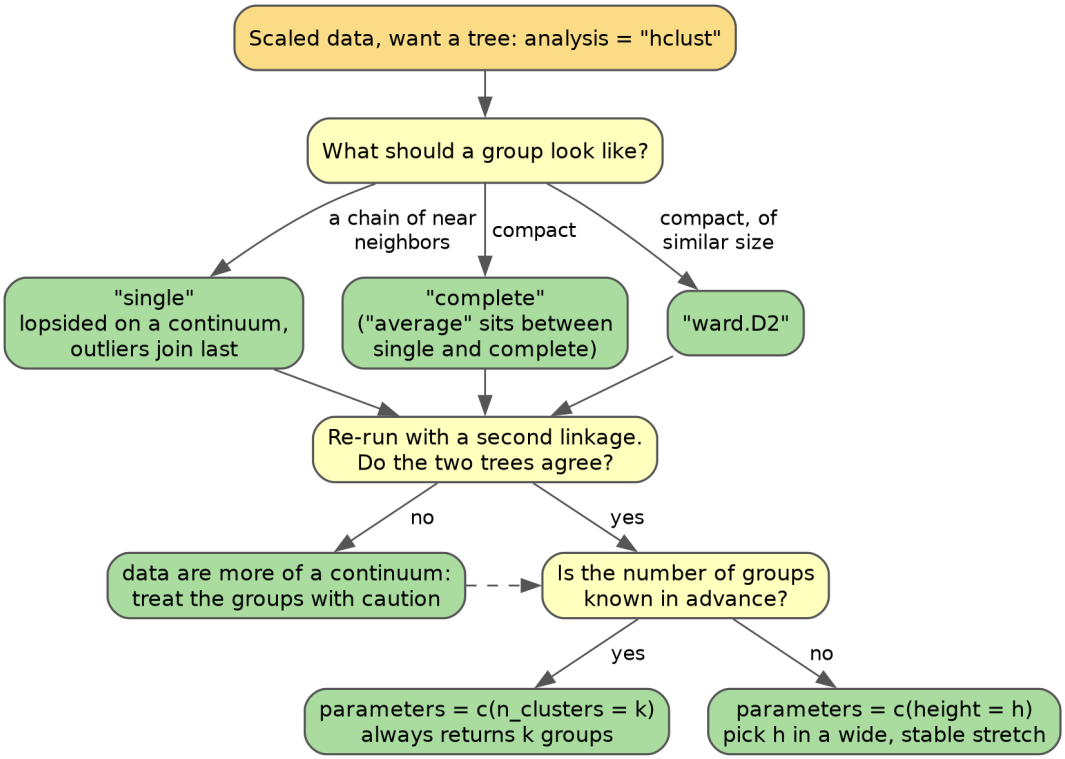


The first decision that must be made when working with networks is determining what an edge means. A **similarity network** is built: distances become similarities, and a chosen threshold decides which pairs are joined, so the network is always undirected and its shape depends on that threshold. An **interaction network** is given by the data themselves: each row is an observed connection, there is no threshold, and the edges may be directed. We use a metric called 'degree' to count a node's connections and a metric called 'strength' adds up a node's traffic. However, neither degree nor strength accounts for whether a node holds different portions of a network together (an articulation point) - only removing the node and counting the pieces that remain can tell us about that. Importantly, the positions of nodes in a force-directed layout carry no quantitative information.

Step	Code from this chapter	What to set
distances, one row per pair	`runMatrixAnalysis(analysis = "dist", output_format = "long")`	`scale_variance`, `columns_w_values_for_single_analyte`, `columns_w_sample_ID_info`, `distance_method`
distance to similarity	`mutate(similarity = 1 / (1 + distance) * 100)`	
threshold	`filter(similarity > 20)`	the cutoff (compare several)
build the network	`buildNetwork()`	`edgelist` (node names in columns 1 and 2, optional weight in 3), `node_attributes`, `directed`
draw it	`geom_segment()` on `net$edges`, `geom_point()` on `net$nodes`	`linewidth` and `alpha` mapped to the weight
degree	`group_by(airport = origin) %>% summarize(out_degree = n())`	origin for out-degree, destination for in-degree
strength	`summarize(passengers_out = sum(passengers))`	
node removal	`filter(origin != "KTN", destination != "KTN")`, then rebuild	the node to remove

hierarchical clustering

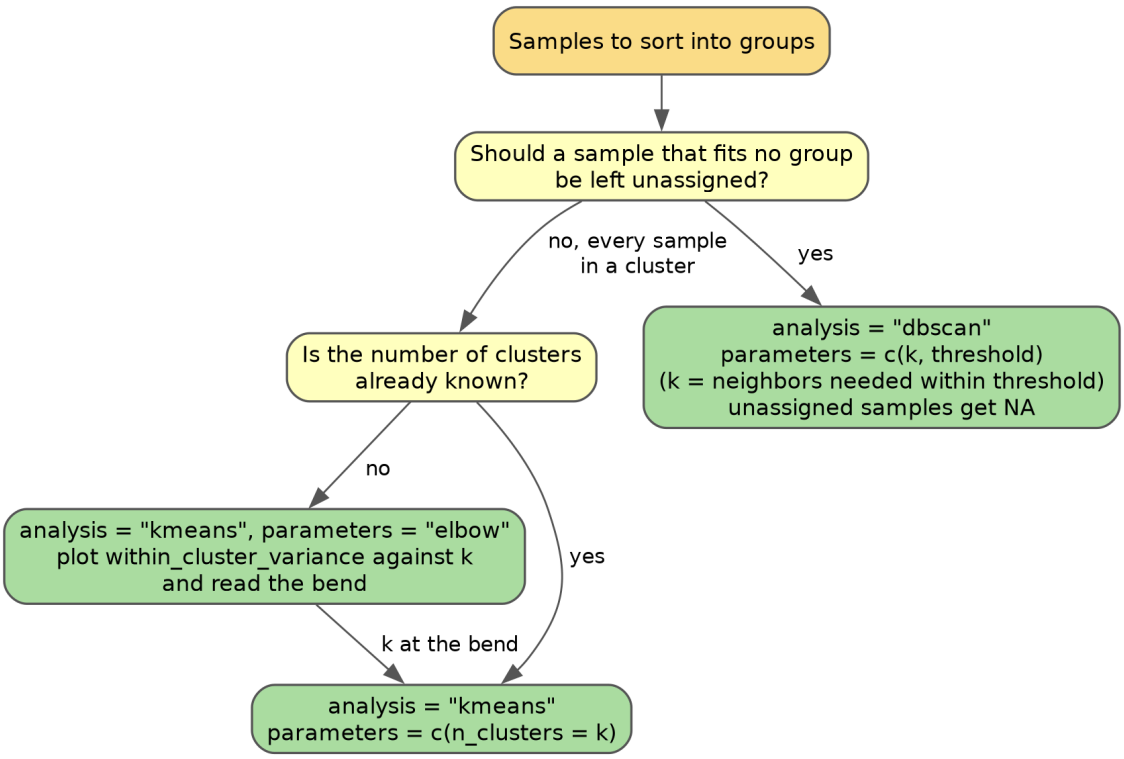
Integrated Bioanalytical Chemistry · thebustalab.github.io/integrated_bioanalytics



Hierarchical clustering merges the two closest groups, again and again, until every sample is connected by one tree structure. **The linkage rule decides what "closest" means for a group**, and different rules build different trees from the same distances. **A tree is read across, not down** (assuming it's drawn with the root on the left and leaves on the right): how far out from the tips two paths meet shows how different two samples are, and adjacent tips are not necessarily similar. **Cutting the tree turns it into groups**: choosing a height as the basis for the cut lets the data drive how many clusters are created, whereas setting a number of groups will force the tree to be cut into that number of clusters.

Function or setting	What it does	What to set or watch
`runMatrixAnalysis(analysis = "hclust")`	builds the tree from the distances	`scale_variance = TRUE`, `agglomeration_method`, `parameters`
`agglomeration_method = "single"`	joins on the closest members	chains a continuum into a lopsided tree
`agglomeration_method = "complete"`	joins on the furthest members	compact groups
`agglomeration_method = "average"`	joins on the average distance	sits between single and complete
`agglomeration_method = "ward.D2"`	adds least spread within groups	compact groups of similar size
`parameters = c(height = 5)`	cuts at a height, adds a `cluster` column	the data decide how many groups
`parameters = c(n_clusters = 4)`	cuts into a set number of groups	always returns that number
`ggtree()` + `geom_tiplab()`, `geom_tippoint()`	draws the tree and labels the tips	`aes(label = lake)`, `aes(fill = park)`, `align = TRUE`
`geom_cut(height = 5)` or `geom_cut(k = 4)`	draws the cut on the tree	the same height as `parameters`, never `geom_vline()`
`filter(isTip)` + `aes(y = y)` in `geom_tile()`	heat map in the tree's tip order	same `scale_y_continuous(limits = ...)` as the tree, then `plot_grid()`

flat clustering

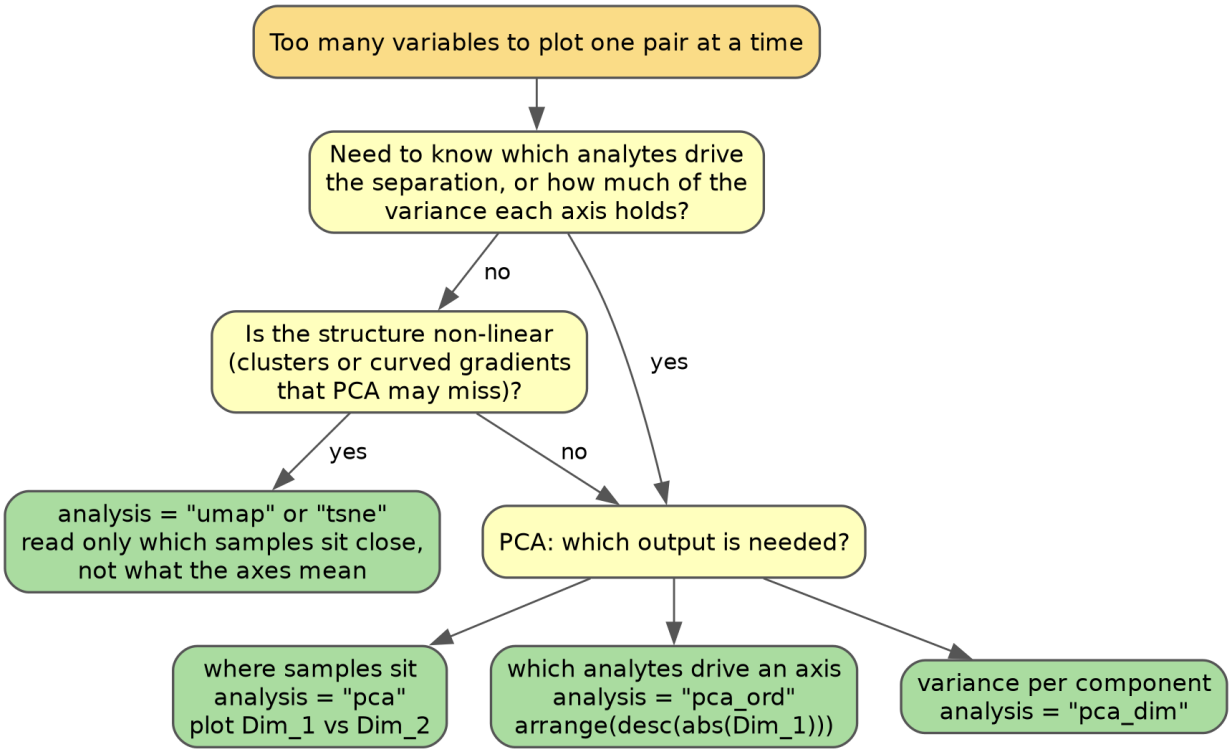


Flat clustering sorts samples straight into groups. **K-means** places every sample in the cluster whose center it sits closest to, so it needs the number of clusters determined ahead of time and can never leave a sample out of a cluster. **Dbscan** grows clusters outwards from points with enough close neighbors and can leave a sample that fits no cluster as `NA`. For k-means, plot the within-cluster variance against k and examine whether there is a bend in the plot (the elbow). A sharp elbow means the number of groups is a property of the data, and a gentle one means that the data are not obviously structured into clusters. Both methods scale the data first by default. Once samples have clusters, the `cluster` column works like any other grouping column.

Function or setting	What it does	What to vary
`runMatrixAnalysis(analysis = "kmeans", parameters = c(n_clusters = 3))`	sorts every sample into the number of clusters asked for	`n_clusters`
`analysis = "kmeans", parameters = "elbow"`	returns k and `within_cluster_variance` for one to ten clusters, for an elbow plot	`c("elbow", 6)` sets a different maximum
`analysis = "dbscan", parameters = c(4, 0.3)`	grows clusters from samples with at least k neighbors within a threshold distance, and leaves the rest as NA	k (the number of neighbors), then the threshold (measured on the scaled columns)
`columns_w_values_for_single_analyte`	names the columns to cluster on	any number of columns
`scale_variance`	scales each column before clustering (TRUE by default)	leave on when columns are in different units
`group_by(cluster) %>% summarize(n = n())`	counts the samples in each cluster (the NA row counts unassigned samples, not a cluster)	other summary statistics
`left_join(x, y, by = "fragment")`	brings back columns that were not clustered on	by, and `select()` the needed columns of y first
`geom_mark_ellipse(aes(fill = cluster, label = cluster))`	draws an ellipse around each cluster	use `drop_na()` on dbscan output to skip the NA samples

dimensional reduction

Integrated Bioanalytical Chemistry · thebustalab.github.io/integrated_bioanalytics

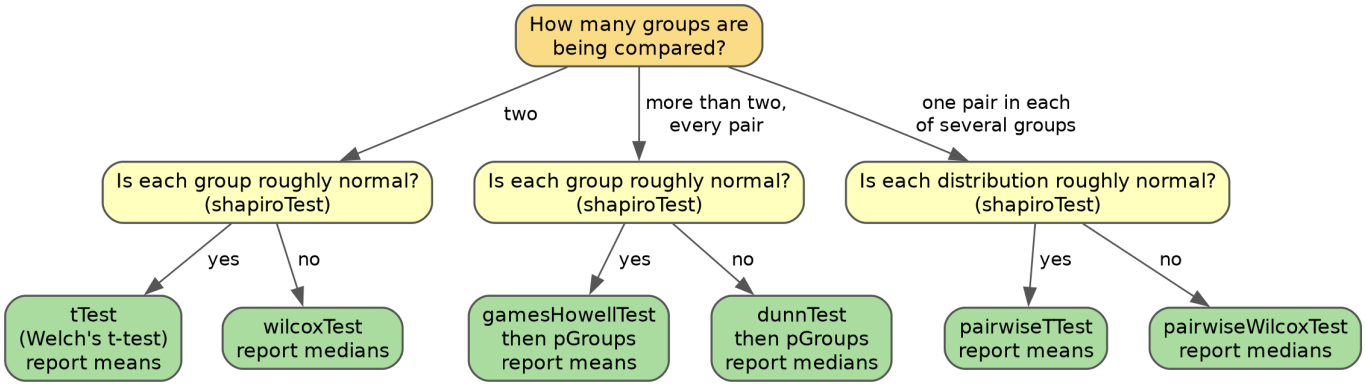


Dimensionality reduction places samples with many measurements on a two-dimensional plot, where nearby samples have similar profiles. **PCA is the one to use when the axes need explaining**: each principal component is a combination of the analytes, the ordination shows which analytes drive each one (ranked by the size of the loading, ignoring its sign), and the scree plot shows how much of the variance each principal component describes. PCA scales every analyte first, and the ordination plot needs `coord_fixed()` so that distances in the two directions can be compared. **UMAP and t-SNE are non-linear** and can reveal clusters or curved gradients that PCA misses, but their axes have no quantitative meaning and each run can come out a little differently, so these should only be used to interpret local structures, not data set-wide layout.

`analysis` or function	What it returns or does	What to set or watch
`"pca"`	scores: one row per sample, with `Dim_1`, `Dim_2`	`columns_w_values_for_single_analyte`, `columns_w_sample_ID_info`, then `coord_fixed()` on the plot
`"pca_ord"`	loadings: one row per analyte	rank with `arrange(desc(abs(Dim_1)))`, draw with `geom_segment()` and `geom_circle()`
`"pca_dim"`	`percent_variance_explained` for each `principal_component`	scree plot, or `filter(principal_component == 1)` for one value
biplot	scores and loadings on shared axes	`geom_point()` from `"pca"` plus `geom_segment()` from `"pca_ord"`
`pcaVisualizer()`	linked scores, ordination and heat map panels	`data` plus the two column arguments, desktop R only
`"umap"`	non-linear embedding	axes have no meaning, runs vary, does not run in browser cells
`"tsne"`	non-linear embedding that favors local clusters	remove samples with duplicate analyte values first, runs vary

comparing groups

Integrated Bioanalytical Chemistry · thebustalab.github.io/integrated_bioanalytics

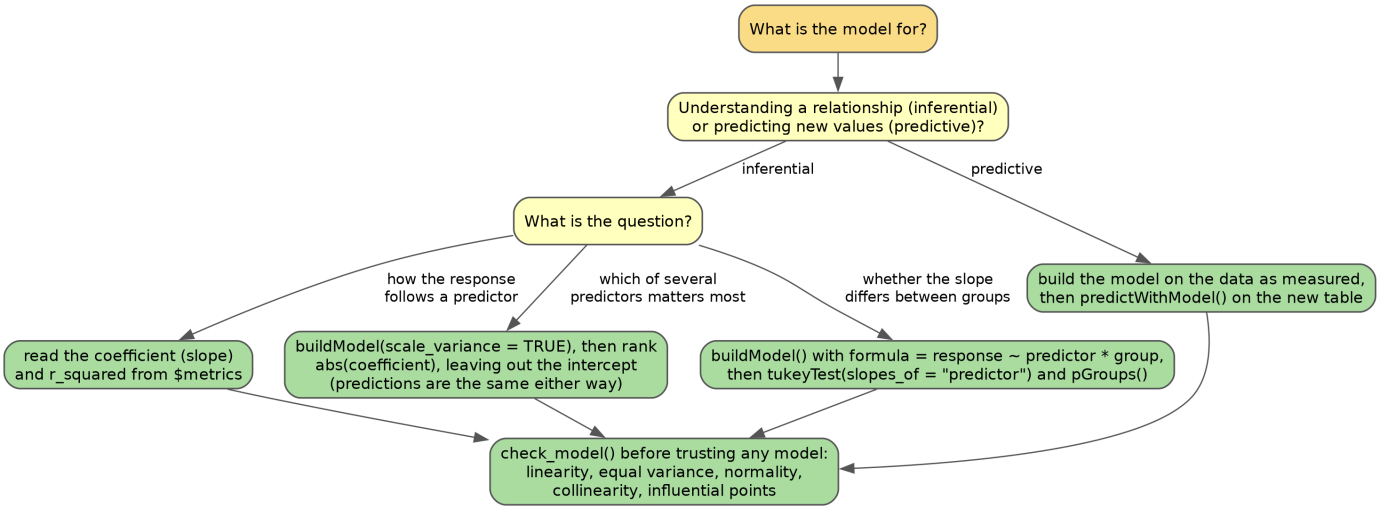


Before choosing a statistical test, look at the distribution of the data in each group. **Can each group be modeled by a normal distribution?** Check with the Shapiro test within each group. If they are not significantly different from normal, use a parametric test. If they are, use a rank test, which asks whether values in one group tend to be higher than values in the other. **How many groups are being compared?** With more than two, a separate t-test on every pair multiplies the chance of a false positive, so the Games-Howell and Dunn tests compare every pair at once and correct the p-values (read `p.adj`). The t-test used in this book is Welch's, and the Games-Howell test and `pairwiseTTest` likewise judge each pair on its own two spreads, so no test in this chapter needs a check for equal variances. The *p* value is the likelihood of getting data as extreme as the data in hand if the distributions they are coming from are in fact the same. The *p* value is not the probability that a detected difference is a false positive. Choosing a test comes down to how many groups are compared and whether the data are normal. Note that ANOVA is addressed in a separate, dedicated chapter.

Function	What it is for	How it is used in this chapter
`shapiroTest(abundance)`	is each group roughly normal? ($p > 0.05$ means not significantly different from normal)	after `group_by()` on the grouping column(s)
`tTest(abundance ~ aquifer_code)`	two groups, roughly normal (Welch's t-test)	`response ~ predictor` formula
`wilcoxTest(abundance ~ aquifer_code)`	two groups, not normal (rank test)	`response ~ predictor` formula
`gamesHowellTest(abundance ~ aquifer_code)`	more than two groups, every pair, roughly normal (each pair judged on its own spreads)	read `p.adj`
`dunnTest(abundance ~ aquifer_code)`	more than two groups, every pair, rank-based	read `p.adj`
`pGroups()`	letters for each group from Games-Howell or Dunn output (a shared letter means no significant difference)	piped after `gamesHowellTest` or `dunnTest`, then drawn with `geom_text()`
`pairwiseTTest(abundance ~ analyte)`, `pairwiseWilcoxTest(abundance ~ analyte)`	one comparison within each of several groups	after `group_by(aquifer_code)`, with `p.adj` corrected across all the groups
`geomSignif()`	draws brackets and stars from pairwise test output	`data` = a table built from the test output

regression models

Integrated Bioanalytical Chemistry · thebustalab.github.io/integrated_bioanalytics

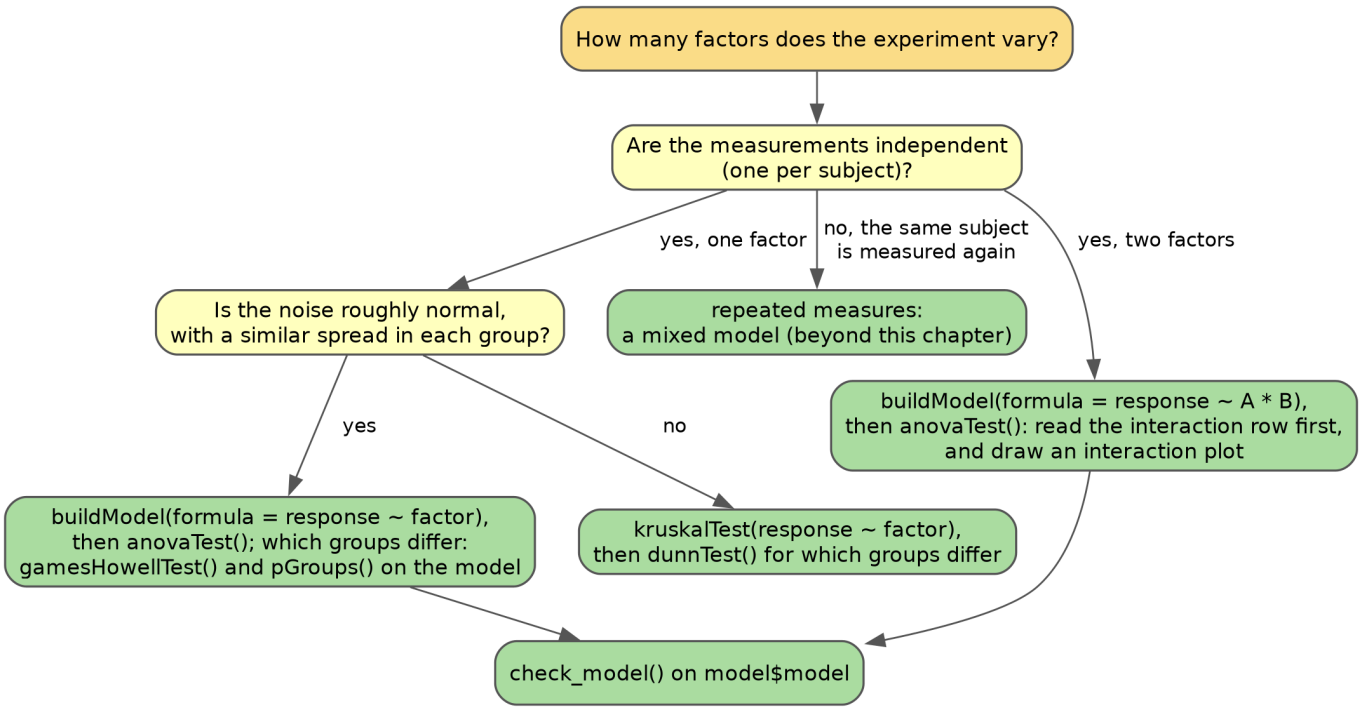


Building a regression model starts with what it is for. An **inferential** use reads the coefficients, their p-values and r-squared to describe how the response relates to the predictors; a **predictive** use applies the model to a table whose response was never measured. Four decisions follow. **How many predictors:** more can explain more of the variation, as r-squared shows. **Whether to scale:** build with `scale_variance = TRUE` before ranking coefficients by size, and leave it off to read them in real units. The fit and the predictions are the same either way. **Whether slopes differ between groups:** join the predictor and the group with `*`, not `+` (one shared slope), then compare the slopes with `tukeyTest(slopes_of = )`. **Whether to trust it:** check the residuals, the predictors and any influential points with `check_model()`.

Function or step	What it is for	What we set or vary
`Hmisc::impute()`	filling in missing values, because `buildModel()` stops on any	the fill value (here each column's median)
`buildModel()`	fitting a model	`data`, `model_type = "linear_regression"`, `predictors` (one or more), `response`; or a `formula`, `response ~ predictor * group`
`$model`, `$metrics`	the fitted model, and a table of its r-squared, coefficients and p-values	`filter(type == "coefficient")`, `filter(type == "statistic")`, `filter(variable == "r_squared")`
`predictWithModel()`	predictions for any table with the same predictor columns, returned as that table with a new `predicted_<response>` column	`data`, and `model` (the whole `buildModel()` output)
`buildModel(scale_variance = TRUE)`	giving the numeric predictors a mean of 0 and a standard deviation of 1 before comparing coefficients, with `predictWithModel()` scaling new data to match	TRUE to rank predictors, FALSE (the default) for coefficients in real units
`arrange(desc(abs(value)))`	ranking coefficients by size	drop the intercept first with `variable != "(Intercept)"`
`tukeyTest(slopes_of = )`, `pGroups()`	which groups' slopes differ in a model built with `*`; letters and each group's slope	`slopes_of`, the numeric predictor whose slope is compared
`check_model()`	diagnostic panels: posterior predictive check, linearity, equal variance, normal residuals, collinearity, influential points	given `model$model`

analysis of variance

Integrated Bioanalytical Chemistry · thebustalab.github.io/integrated_bioanalytics



An ANOVA splits the spread in a response (its sum of squares) into the part each term explains and the part left over, the noise. **Build the model, then ask it questions:** fit it once with a formula (response on the left, factors on the right, ∗ for an interaction), read the ANOVA table, then, for a single factor, use the Games-Howell test on the same model to find which groups differ. **Read the table row by row:** each row's free values are what is left once the values the model fixes are spoken for (the grand mean speaks for one measurement, a factor with *k* groups keeps *k* – 1, an interaction multiplies its terms', and the noise gets the rest). A row's variance is its sum of squares per free value, F is that variance over the noise variance, so it is evidence that a term matters, and `share` is how much: the row's share of the variation explained, with every row's share adding up to 1. **With two factors, read the interaction row first:** if it is real, a main effect alone can mislead, and an interaction plot shows how. **Check the model** before trusting it: noise roughly normal with a similar spread in every group, and measurements independent (repeated measurements of one subject need a mixed model). For a single factor with non-normal data, use the Kruskal test, then the Dunn test.

Function	What it is for	How it is used in this chapter
`buildModel(formula = )`	fitting the model the ANOVA is read from	`response ~ factor`, or `response ~ factor_1 * factor_2` for an interaction, with `model_type = "linear_regression"`
`anovaTest()`	the ANOVA table: a row per term, then noise and total, with `sum_of_squares`, `free_values`, `variance`, `F`, `p` and `share`	piped after the model
`gamesHowellTest()`, `pGroups()`	which groups differ, as letters, using each group's own spread	piped after the same one-factor model
`kruskalTest(abundance ~ aquifer_code)`	rank-based test of any difference, one factor	on the data, followed by `dunnTest()`
`check_model()`	diagnostic panels for the noise (residuals)	given `model$model`

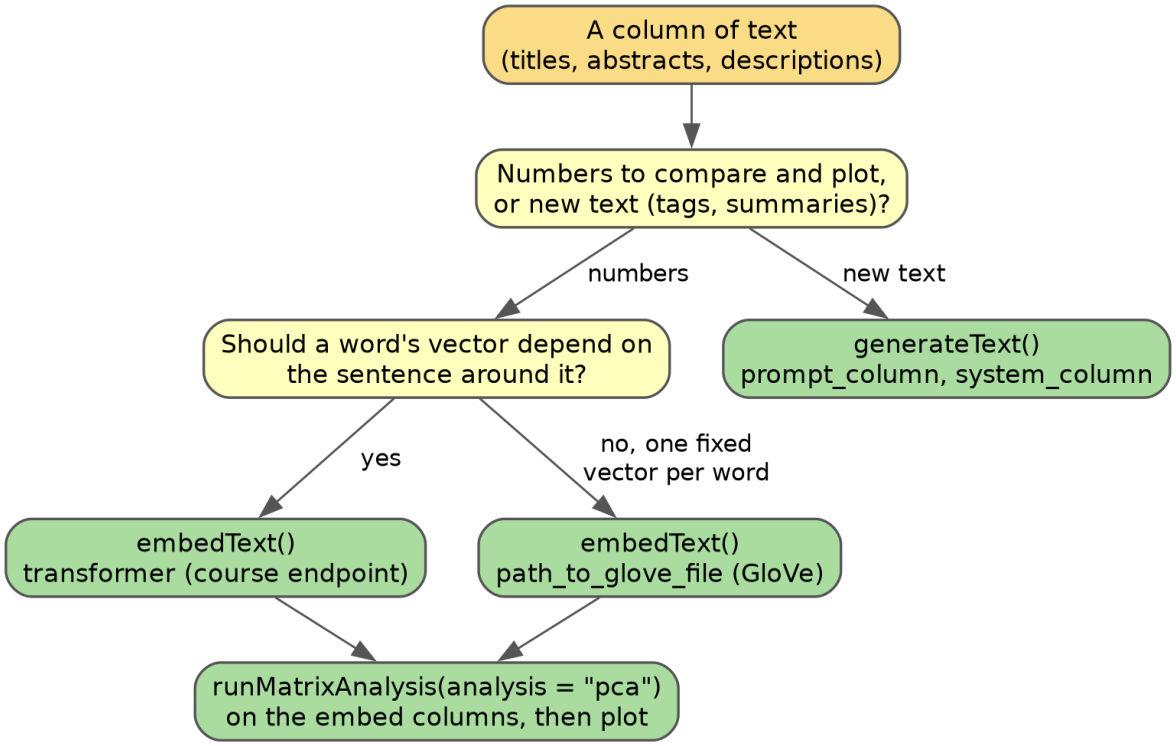
machine learning models

Flexible models are judged by how well they predict data they have not seen, because a model that is too complex fits the noise in its training data. A **random forest** builds many decision trees, each from a random subset of the data and trying a random subset of the predictors at each split, then averages their predictions (regression) or takes a majority vote (classification). The kind of response decides the `model_type`. `buildModel()` tries every combination of settings in the `optimization_parameters` grid and reports each one's cross-validated metric: RMSE for regression, where lower is better, and accuracy for classification, where higher is better. `predictWithModel()` then applies the best model to new samples.

Function or setting	What it is for	What we set or vary
`model_type = "random_forest_regression"`	a forest that predicts a number, such as Valine abundance	used with a numeric `response`
`model_type = "random_forest_classification"`	a forest that predicts a category, such as `patient_status`	used with a categorical `response`
`buildModel()`	building and tuning the forest	`data`, `model_type`, `predictors`, `response`, `optimization_parameters`
`optimization_parameters`	the grid of settings to try, as a `list()` of `seq()` ranges	the three settings below
`n_vars_tried_at_split`	how many predictors are candidates at each split	smaller values make the forest more random
`n_trees`	how many trees the forest grows	more trees give steadier predictions, with diminishing returns
`min_leaf_size`	the fewest samples a node needs before it is split again	larger values make simpler, less overfitted trees
`$metrics`	one row per combination, with `mean` holding its cross-validated `.metric`	for a classifier, sort with `arrange(desc(mean))` (higher accuracy is better), or map it across the grid with `geom_tile()`
`predictWithModel()`	predictions for samples the model was not built on, returned as the same table with `predicted_<response>` (regression) or `probability_` and `predicted_class` columns (classification) added	`data`, and `model` (the whole `buildModel()` output)

language models

Integrated Bioanalytical Chemistry · thebustalab.github.io/integrated_bioanalytics



Language models turn text into numbers. An embedding model converts each title, abstract or description into a vector (384 numbers for the course transformer model), and texts with similar meanings get similar vectors, so familiar tools such as PCA can show which ones group together. The two embedding methods differ in how they treat context: GloVe gives every word one fixed vector and averages the word vectors over a sentence, whereas a transformer adjusts each word's vector to fit the sentence around it. A generative model builds on the same machinery to write new text in response to a prompt and a system message, and it is much slower and more expensive than embedding because each row is a separate request to a large model. Keys and tokens live in `.Renviron` and are read with `Sys.getenv()`, never typed into code.

Function	What it is for	Arguments varied in this chapter
`searchPubMed()`	retrieve PubMed records (title, abstract, journal) for a set of search terms	`search_terms`, `pubmed_api_key = Sys.getenv("PUBMED_TOKEN")`, `retmax_per_term`, `sort` (`"relevance"` or `"date"`)
`embedText()`, transformer	contextual embeddings from the course endpoint (BAAI/bge-small-en-v1.5, 384 columns)	`df`, `column_name` (the class token is picked up from `.Renviron`)
`embedText()`, GloVe	one fixed vector per word, averaged over each sentence	`column_name`, `path_to_glove_file = Sys.getenv("GLOVE_FILE")`
`runMatrixAnalysis()`	PCA on the embedding columns to see which texts group together	`analysis = "pca"`, `columns_w_values_for_single_analyte` (the columns found by `grep("embed", ...)`), `columns_w_sample_ID_info`
`generateText()`	new text (here, three tags per abstract) from the course generation endpoint	`df`, `prompt_column`, `system_column`

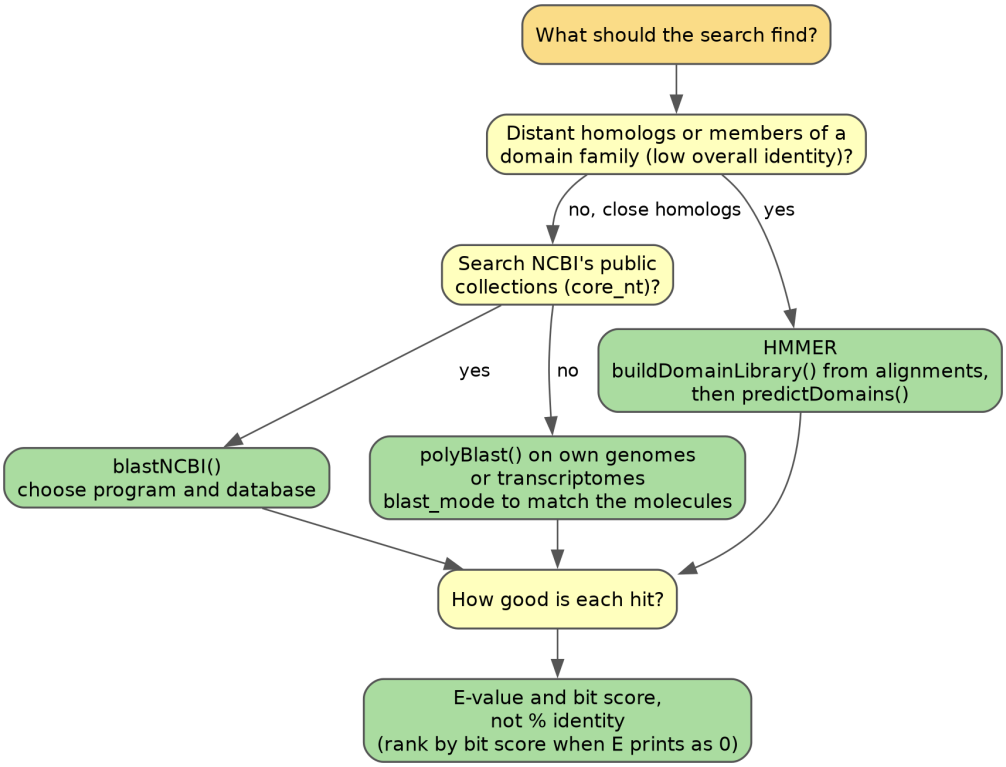
protein language models

A protein language model turns each amino acid sequence into an embedding, a row of numbers reflecting patterns the model learned from vast numbers of protein sequences. Proteins with similar embeddings tend to share evolutionary, structural or functional features, and because a transformer's attention mechanism relates every amino acid to every other, the embedding can reflect residues that are far apart in the sequence but close in the folded protein. Unlike a multiple sequence alignment, an embedding needs no homologous sequences and is fast to compute once the model is trained. The workflow has three steps: retrieve sequences, embed them, and run a PCA on the embedding columns to see whether proteins with different functions separate.

Step	Function	What it is for	Arguments and settings used in this chapter
1	`searchNCBI()`	retrieve protein sequences from NCBI as an AAStringSet	`search_term`, `retmax`
1	(subsetting)	keep sequences short enough to embed whole, since esm2 truncates long sequences (the chapter keeps those under 1024 amino acids)	`all_sequences[all_sequences@ranges@width < 1024]`
2	`embedAminoAcids()`	embed each sequence with a pre-trained protein language model	`amino_acid_stringset`, `platform` (`"nvidia"` runs esm2-650m, bioLM offers several models), `nvidia_api_key` or `biolm_api_key`
3	`runMatrixAnalysis()`	PCA on the embedding columns to see which proteins group together	`analysis = "pca"`, `columns_w_values_for_single_analyte` (the embedding columns), `columns_w_sample_ID_info`

homology

Integrated Bioanalytical Chemistry · thebustalab.github.io/integrated_bioanalytics



A homology search starts with two choices. **BLAST or HMMER**: BLAST is the fast first pass for close homologs and rapid annotation, while HMMER profiles built from alignments are more sensitive for distant homologs and shared domains. **Where to search**: `blastNCBI()` sends one sequence to NCBI's servers, and `polyBlast()` searches our own genomes or transcriptomes with many queries at once, with a `blast_mode` that matches the molecule types. Hits are then judged by E-value and bit score rather than percent identity, and ranked by bit score once E-values become so small that R prints them as zero.

Function	What it does	Arguments students vary
`blastNCBI()`	remote BLAST of one sequence on NCBI's servers, returning a tibble of hits	`query_sequence`, `program` (`"blastn"`, `"megablast"`), `database` (`"core_nt"`)
`polyBlast()`	local BLAST+ search of one or more queries against several sequence collections, writing a hit list and one file per hit (returns nothing)	`named_subjects_list`, `query_in_path`, `blast_mode`, `e_value_cutoff` (default 1), `queries_in_output`, plus the paths `sequences_of_interest_directory_path`, `monolist_out_path`, `blast_module_directory_path`
`buildDomainLibrary()`	trains profile HMMs from multiple sequence alignments into a domain library	`alignment_in_paths`, `domain_library_out_path`
`predictDomains()`	scores sequences against a domain library to find likely family members	`fasta_in_path`, `domain_library_in_path`

Query	Subjects	`blast_mode`
nucleotide	nucleotide	`"nnblastn"` (classic blastn)
protein	nucleotide, translated	`"ntblastp"` (like tblastn)
protein	protein	`"pnblastp"` (like blastp)

alignments

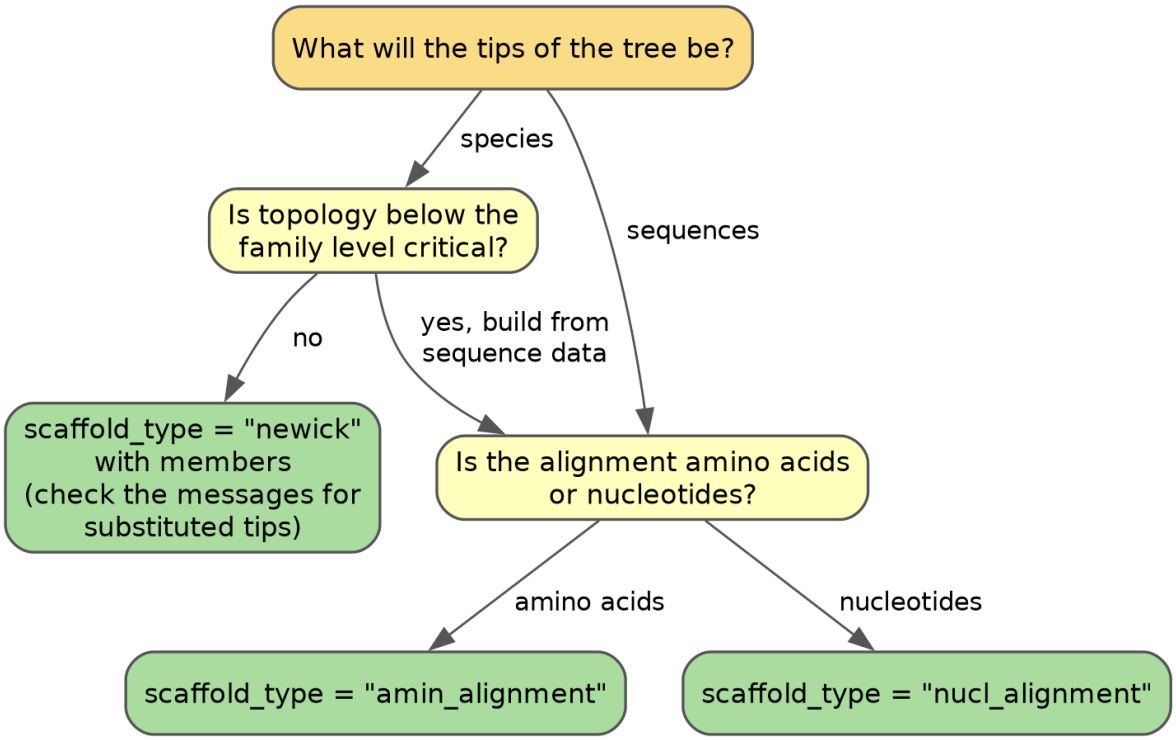
Integrated Bioanalytical Chemistry · thebustalab.github.io/integrated_bioanalytics

Building an alignment takes two steps. `alignSequences()` aligns the hits chosen by a logical `subset` column of the `polyBlast()` monolist, with `mode` set to nucleotide, amino-acid or codon alignment. `analyzeAlignment()` then profiles each column for gaps and conservation, lets us tune the trimming thresholds interactively, and writes the trimmed alignment to `<alignment_in_path>_trimmed`, ready for tree building.

Function	What it does	Arguments students vary
`readMonolist()`	reads the hit-list CSV written by `polyBlast()`	the CSV path
`alignSequences()`	aligns the monolist rows where the `subset` column is TRUE and writes an aligned FASTA	`subset`, `input_sequence_type` (`"nucl"`, `"amin"`), `mode` (`"nucl_align"`, `"amin_align"`, `"codon_align"`, `"fragment_align"` with `base_fragment`), `alignment_out_path`, `sequences_of_interest_directory_path`
`analyzeAlignment()`	Shiny app for trimming gap-heavy or poorly conserved columns, returning the trimmed alignment	`alignment_in_path`, `type` (`"DNA"`, `"AA"`), `jupyter` (TRUE in Jupyter)

phylogenies

Integrated Bioanalytical Chemistry · thebustalab.github.io/integrated_bioanalytics



Building a tree starts with one decision: **what the tree is built from.** A newick scaffold is quick for species when only the relationships above the family level matter, and `buildTree()` grafts a species missing from the scaffold onto a relative's tip, so its messages are worth reading. When genus-level topology matters, or when the tips are genes, the tree should come from a sequence alignment instead, by neighbor joining or by maximum likelihood. Once built, a tree is plotted with `ggtree()`, joined to data after `fortify()`, collapsed to a higher level with `collapseTree()`, and drawn beside its traits, where the trait axis must be reordered to match the tree and the tree tips and trait labels checked by eye to confirm they line up.

Function	What it is for	Settings students vary
`buildTree()` from a newick scaffold	a tree of species pruned from an existing phylogeny	`scaffold_type = "newick"`, `scaffold_in_path`, `members`
`buildTree()` from an alignment	a tree built from aligned sequences	`scaffold_type` (`"amin_alignment"` or `"nucl_alignment"`), `scaffold_in_path`, `ml` (TRUE for maximum likelihood, FALSE for neighbor joining), `model_test`, `bootstrap`, `ancestral_states`, `root`
`plot()`	a quick look at a small tree	none
`ggtree()`	plotting a tree with ggplot layers	add `geom_tiplab()` with `offset`, `align`, `size`
`fortify()` + `left_join()`	turning a tree into a data frame and adding metadata by tip label	`by = c("label" = ...)`
`collapseTree()`	one tip per higher group (for example, per family)	`tree`, `associations` (tip labels, then group)
`reorder(label, y)`	putting trait rows in the same order as the tree tips	the column to reorder and the tree's y
`plot_grid()`	the tree and the trait plot side by side	`nrow = 1`, `align = "h"`, `axis = "tb"`

phylogenetic analyses

The four analysis functions in this chapter take the same inputs: a tree as a `phylo` object, and a long table of traits with one column of species names that match the tree's tip labels exactly. Choosing among the analyses is a matter of choosing the question. **Phylogenetic signal** asks whether related species share similar trait values. **Independent contrasts** remove the effects of shared ancestry before two traits are compared. **Ancestral trait reconstruction** estimates the trait values of ancestors at the tree's nodes. It is not the same as `buildTree()`'s `ancestral_states` in the phylogenies chapter, which computes ancestral sequences.

Function	Question it answers	Settings students vary
`runPhylogeneticAnalyses()`	the comparative analyses in one call	`traits`, `column_w_names_of_tiplabels`, `column_w_names_of_traits`, `column_w_values_for_traits`, `tree`
`phylogeneticSignal()`	do close relatives have similar trait values?	the same five arguments
`independentContrasts()`	are two traits related once shared ancestry is removed?	the same five arguments
`ancestralTraits()`	what trait values did the ancestors have?	the same five arguments
`geom_ancestral_pie()`	pie charts of the ancestral estimates at each node of a `ggtree()` plot	`data = filter(anc_traits_tree, isTip == FALSE)`, `pie_size`, `pie_alpha`

The long trait table is made with `pivot_longer()`, and a tree passed to `ggtree()` for ancestral pies first needs duplicate nodes removed with `distinct(anc_traits_tree, node, .keep_all = TRUE)`.